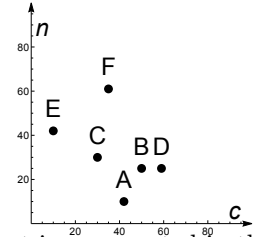


A. Ranking (9 points)

The following table reports data about Air Conditioning appliances, including cost (c), noisiness (n) and probability of failure during the first year (p).

	c	n	p
A:	42.0	10.0	93.0
B:	50.0	25.0	44.0
C:	30.0	30.0	50.0
D:	59.0	25.0	45.0
E:	10.0	42.0	20.0
F:	35.0	61.0	25.0



1. Find the **skyline** of the dataset using SFS.
2. Remove the p dimension and determine the skyline again via SFS (the new dataset is represented in the figure above). State how many times you had to test dominance between appliances to find the result.
3. For each appliance in the skyline of Question 2, show a scoring function that makes it top-1 (in the dataset without p). If this is impossible, explain why.

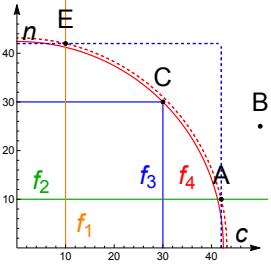
Solution.

1. We use SFS. It suffices to sort the data by their c value (there are no ties):

	c	n	p
E:	10.0	42.0	20.0
C:	30.0	30.0	50.0
F:	35.0	61.0	25.0
A:	42.0	10.0	93.0
B:	50.0	25.0	44.0
D:	59.0	25.0	45.0

The skyline points are E, C, A, B, since F is dominated by E and D by B.

2. Without p , the skyline points are E, C, A. Now, B is dominated by A. Clearly, F continues to be dominated by E and D by B. Ten dominance tests are needed in total by SFS: $E \not\prec C$, $E \prec F$, $E \not\prec A$, $C \not\prec A$, $E \not\prec B$, $C \not\prec B$, $A \prec B$, $E \not\prec D$, $C \not\prec D$, $A \prec D$, where $\prec$ indicates dominance.
3. Every skyline tuple necessarily ranks first for some scoring function. The function $f_1(t) = t.c$ makes E top-1, since E has the lowest c value in the dataset; similarly, $f_2(t) = t.n$ makes A top-1. No linear function can make C the top-1 tuple; yet, C ranks first according to $f_3(t) = \max\{t.c, t.n\}$; another function that makes C the top tuple is $f_4(t) = t.c^2 + t.n^2$, as $f_4(C) = 1800 < f_4(E) = f_4(A) = 1864$. The figure offers a close-up of the dataset, showing iso-score curves for E using f_1 (orange), for A using f_2 (green), for C using f_3 (blue) and f_4 (red); the iso-score curves for E and A using f_3 and f_4 are shown as dashed curves.



B. Physical Databases (7 points)

Table `ClassAssignment(Year, Student, Class, Section)` for each year, associates students with a class and section (e.g, in 2022, Bob is in class 1 section A). Data are stored in a primary B+ tree on `(Year, Student)` with 3 levels and 100 leaf nodes, with 10K tuples. The primary key is underlined.

Table `Class(Year, Class, Section, Classroom, Notes)` records the classroom of the school assigned to each class-section every year (e.g., in 2022, 1 A is assigned to Room1). This is primarily entry-sequenced on 50 blocks and contains 500 tuples.

Describe an efficient query plan for the query below, knowing that $\text{val}(\text{Year})=20$ for both tables and $\text{val}(\text{Classroom})=25$. Evaluate its cost in the following two scenarios:

1. No auxiliary structures.

2. Table `Class` has a hash index on `(Year, Class, Section)` occupying 10 blocks, with no overflow chains.

```
SELECT Student FROM Class C JOIN ClassAssignment CA ON C.Year=CA.Year AND C.Class=CA.Class AND C.Section=CA.Section WHERE Classroom="Room104";
```

Solution.

Scenario 1

Option 1: Start from `ClassAssignment`. The visit to all the leaf nodes costs 2 (visit two intermediate nodes) + 100 leaf nodes. Here we can get the student to be returned in the select clause. In these nodes we find 10K tuples: for each tuple, we find the corresponding tuples in the `Class` table to check the classroom, by scanning the whole primary structure that consists of 50 blocks.

$C = 2 + 100 + 10K * 50 = 500K$ I/O.

The cost is too expensive!

Option 2: Start from `Class` where you can filter the tuples that satisfy the `WHERE` condition. The primary structure of `Class` is scanned to find the tuples for Classroom "Room104". The cost to scan it is 50 blocks. The number of tuples for Classroom "Room104" is $20 = \text{total number of tuples} / \text{val}(\text{Classroom}) = 500/25$. For each tuple we need to find the tuples in `ClassAssignment` that satisfy the join condition to retrieve the student data to be returned in the select clause. One possible way is: given the year in each Classroom "Room104" tuple, access the B+; this requires to visit 2 intermediate nodes, then, in the leaf nodes, we need to visit all the leaf nodes for that year. Among the 10K tuples in `ClassAssignment`, we have $10K/\text{val}(\text{Year}) = 10K/20 = 500$ tuples for each year. One leaf node contains on average $10K \text{ tuples} / 100 \text{ leaf nodes} = 100$ tuples. Therefore, 500 tuples (for one year) require 5 leaf nodes. Since the first tuple of a year will not start exactly at the beginning of a leaf node, in general we need to read an additional leaf node. For each year, we therefore need to read 2 intermediate blocks + (5 + 1 extra) leaf nodes.

Cost: $50 [\text{scan Class}] + 20 [\text{tuples "Room104"}] * (2 + 5 + 1) [\text{for each year access the ClassAssignment leaf nodes}] = 210$ I/O.

Option 3. Follow the initial steps like in Option 2 to identify the tuples with classroom "Room104", which are 20.

These 20 tuples can be cached. All the leaf nodes (100 blocks) in the B+ can be read and compared with the 20 cached tuples to check the join conditions and return the student data.

Cost: $50 [\text{scan Class and cache 20 tuples}] + 2 [\text{intermediate nodes}] + 100 [\text{scan ClassAssignment}] = 152$ I/O

Scenario 2

The added hash index can improve Option 1. Instead of scanning the whole primary structure, for each tuple of `ClassAssignment`, we can access the tuples of a given `(Year, Class, Section)` via the index. `(Year, Class, Section)` is the primary key, so in the index there will be exactly one link to the tuple in the primary structure. For each tuple of `ClassAssignment` we therefore need to access one block of the hash index (there is no overflow chain) + one block in the primary structure.

The cost of the improved version of Option 1 becomes:

$C = 2 + 100 + 10K * (1 + 1) \approx 20K$ I/O,

which is still very expensive.

In conclusion, for both scenarios, the best solutions are options 2 and 3. The other solutions have a much higher cost.

C. Trigger (9 points)

A parcel delivery company manages lockers distributed in various locations in major cities. A locker has N cells. Parcels can be small or large. A cell can store either one large parcel or up to four small parcels (but not both kinds of parcels at the same time). A parcel can be put in one cell by an employee of the company, moved from one cell to another cell by an employee of the company, and picked up by a user.

The relational schema of the tables is:

```
LOCKER(locker_id, city, street, number, zipcode, lat, long) -- locker_id is PRIMARY KEY
CELL(cell_id, locker_id, numlarge, numsmall) -- cell_id is PRIMARY KEY
PARCEL(parcel_id, cell_id, size, descr, image) -- parcel_id is PK, size is VARCHAR
```

When a locker is created also its cells are created. Lockers and cells cannot be deleted. The numlarge, numsmall attributes of a cell default to 0. When a parcel is created, the cell_id is initially null. When a parcel is deposited in a locker, the cell_id attribute is updated to reflect the location where the parcel is stored. When a parcel is picked up, the tuple is deleted from the PARCEL relation.

Write the triggers that manage the update of the numlarge and numsmall attributes of a cell after the insertion, pickup and repositioning of a parcel, so as to ensure consistency of the data.

Solution.

```
CREATE TRIGGER update_parcel_before_insert_or_shift
BEFORE UPDATE on parcel
for each row
when (old.cell_id is null and new.cell_id is not null) or old.cell_id <> new.cell_id
-- insertion or repositioning
begin
    if not exists(select * from cell where cell_id = new.cell_id) -- target cell exists
    or (1 = (select numlarge from cell where cell_id = new.cell_id)) -- there is space
    or (4 = (select numsmall from cell where cell_id = new.cell_id)) -- there is space
    or (new.size = 'large' and 0 < (select numsmall from cell where cell_id = new.cell_id))
    -- there is space
    then
        raise exception 'Cell does not exist or there is not enough space for the parcel!';
    end if;
end;

CREATE TRIGGER update_parcel_after_insert_or_shift
AFTER UPDATE on parcel
for each row
when (old.cell_id is null and new.cell_id is not null) or old.cell_id <> new.cell_id
-- insertion or repositioning
begin
    if new.size = 'small' then
        update cell set numsmall = numsmall + 1 where cell_id = new.cell_id;
        if old.cell_id is not null then
            update cell set numsmall = numsmall - 1 where cell_id = old.cell_id;
        end if;
    else
        update cell set numlarge = numlarge + 1 where cell_id = new.cell_id;
        if old.cell_id is not null then
            update cell set numlarge = numlarge - 1 where cell_id = old.cell_id;
        end if;
    end if;
end;
```

```
CREATE TRIGGER delete_parcel_after_pickup
AFTER DELETE on parcel
for each row
when old.cell_id is not null
begin
    if old.size = 'small' then
        update cell set numsmall = numsmall - 1 where cell_id = old.cell_id;
    else
        update cell set numlarge = numlarge - 1 where cell_id = old.cell_id;
    end if;
end;
```

D. JPA (7 points)

Given the database of the parcel delivery company of exercise C, an application permits the employee to access the list of the lockers in one city, access the list of the cells of the locker, check the status of occupancy and the details of the parcels contained. Lockers are in the order of thousands and cells per lockers are in the order of tens.

Draw the Entity-relationship model corresponding to the relational model, show the JPA entities (with all their attributes) that map the domain objects of the relational model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.

Solution.

See separate pdf file.